Practical Exam: SWD392 – SP26

Duration: 85 Minutes

Instructions

- Using the Internet during the exam is **NOT** allowed.
- You may use **UML modeling tools in offline mode** (such as Visual Paradigm, Visio, Draw.io).

Business Overview

FPT University regularly hosts a range of academic and extracurricular activities, including seminars, workshops, technology talks, student competitions, and orientation programs.

To support event planning and execution, an **Event Management System_EMS** has been developed. Through this system, organizers can create and manage events, register participants, assign staff members, and track event status.

Students and lecturers may register to attend events, while event organizers and administrators are responsible for approving events, managing registrations, and monitoring event execution.

Each event has a defined schedule, location, and capacity. The system automatically tracks participant registration status and updates event states throughout its lifecycle.

This exam is designed to assess your ability to design software architecture and UML models rather than to implement program logic.

The examination evaluates students' understanding of software architecture and UML-based system design. Students are **not required to write any source code**. All answers must be presented using UML diagrams.

Students may choose either **Layered Architecture** or **Microservices Architecture**, but the chosen architecture must be applied consistently throughout all diagrams. The **Repository Pattern is mandatory**.

Where necessary, students may make reasonable assumptions, provided that those assumptions do not contradict the given business context.

Question 1 (3.0 points) – Class Diagram

You must design a design-level Class Diagram for the University Event Management System EMS.

The system involves multiple user roles with shared attributes and specialized responsibilities. Organizers create and manage events, while students and lecturers register to participate.

Each event has a limited capacity and a defined lifecycle. Participant registrations must be tracked and managed throughout the event process.

To keep the architecture clean and maintain separation of concerns, business logic is handled by service classes, while data access is handled by repository classes.

Your Class Diagram must satisfy the following requirements:

- User is a base class.
- Student, Lecturer, Organizer, and Admin are specialized types of User using inheritance.
- An Event represents a university-organized activity.
- A Registration represents a user's participation in an event.
- A user can register for multiple events.
- Each event can have multiple registrations but has a limited capacity.
- Each event has a status (e.g., Planned, OpenForRegistration, Ongoing, Completed, Cancelled).
- Business logic related to event creation, registration, and management is handled by service classes.
- Repository classes must implement a generic Repository interface.
- The service layer depends on repository classes for data access.

*— Tài liệu được thành viên đăng tải tại **FUOVERFLOW** và được biên soạn lại để hỗ trợ ôn tập —*

The following UML relationships must be shown clearly:

- Inheritance and interface implementation
- Association, aggregation, and composition where appropriate
- Dependency relationships between service and repository layers
- Clearly defined attributes, methods, and multiplicities

Question 2 (4.0 points) – Sequence Diagram

Use Case: Register for Event

This question focuses on the interaction flow when a student or lecturer registers for an event.

The process starts when the user selects an event and submits a registration request. The system then checks whether the event is open for registration and whether capacity is still available.

If the registration conditions are satisfied, the system creates a registration record and confirms participation; otherwise, the registration is rejected.

Your Sequence Diagram must meet the following requirements:

- The diagram must follow **either Layered Architecture or Microservices Architecture** consistently.
- The interaction flow must clearly include:
  - A user-facing component (UI / boundary, e.g., Student Portal)
  - A controller or service component handling business logic
  - Repository components responsible for data access
  - Domain entities such as `Event` and `Registration`
- Repository Pattern usage must be explicit through repository method calls.
- Conditional behavior must be modeled using alt fragments for:
  - Event open or closed
  - Event capacity available or full

Question 3 (3.0 points) – State Diagram

Target: Event Lifecycle Management

Each event at FPT University goes through multiple states from creation to completion or cancellation. The system must track event status to ensure proper organization and participant management.

You must design a **State Diagram** for the `Event` entity that describes the complete lifecycle of an event.

Your State Diagram must satisfy the following requirements:

- The diagram must include a clearly defined **initial state** and **final state**.
- The diagram must include at least the following states (state names may vary, but meaning must be preserved):
  - Draft – event is created but not yet published
  - Approved – event is approved by an administrator
  - OpenForRegistration – participants can register
  - Full – maximum capacity reached
  - Ongoing – event is currently taking place
  - Completed – event has ended successfully
- The diagram must include an alternative termination state such as:
  - Cancelled
  - Rejected
- At least **six state transitions** must be shown, each with a clear triggering event, such as:
  - submitEvent
  - approveEvent
  - openRegistration
  - registerParticipant
  - startEvent
  - finishEvent
  - cancelEvent
- At least one branching path must represent **event cancellation or rejection**.

END OF EXAMINATION

(Invigilators will not provide further explanations)